

Assignment: Comparative Study of Shallow CNN vs Deep CNN on Fashion-MNIST

Short Comparative Report

Submitted By: Vaibhav Chavan

Course-Batch: PGDSAIML_B1

Table of Contents

1. Objective	3
2. Dataset Overview	3
3. Part 1: Load and Explore the Dataset	4
4. Part 2: Build and Train a Shallow CNN	5
5. Part 3: Build and Train a Deep CNN	6
6. Part 4: Comparative Study of Shallow CNN vs Deep CNN	8
7. Part 5: Prediction and Error Analysis	10
8. Part 6: Final Comparative Conclusion	12
9. Final Conclusion	13

1. Objective

The objective of this study is to perform a comparative analysis between a Shallow Convolutional Neural Network (CNN) and a Deep CNN using the Fashion-MNIST dataset. Both models are designed, trained, and evaluated under identical conditions to compare their classification performance, generalization ability, computational efficiency, and error patterns. The study aims to determine which architecture is more suitable for this image classification task and to understand the trade-offs between model complexity and performance.

Specifically, this report addresses the following objectives: (1) loading and exploring the Fashion-MNIST dataset, (2) building and training a Shallow CNN with 2 convolutional layers, (3) building and training a Deep CNN with 4 convolutional layers, (4) performing a side-by-side comparative study of both models, (5) conducting prediction and error analysis with confusion matrices, and (6) drawing a final comparative conclusion with recommendations.

2. Dataset Overview

The Fashion-MNIST dataset is a widely used benchmark dataset created by Zalando Research as a drop-in replacement for the original MNIST handwritten digit dataset. It consists of 28x28 grayscale images of clothing items, with a training set of 60,000 examples and a test set of 10,000 examples. Each image is associated with a label from one of 10 classes.

Fashion-MNIST Class Labels

Label	Class Name	Description
0	T-shirt/top	Short-sleeved upper garment
1	Trouser	Full-length lower garment
2	Pullover	Long-sleeved knit upper garment
3	Dress	One-piece garment
4	Coat	Outerwear upper garment
5	Sandal	Open-toed footwear
6	Shirt	Button-front upper garment
7	Sneaker	Closed-toed athletic footwear
8	Bag	Handbag or accessory
9	Ankle boot	Ankle-height footwear

The dataset is balanced, with each class containing exactly 6,000 training images and 1,000 test images. This balanced distribution ensures that no class is over- or under-represented during training, which simplifies the evaluation process and allows accuracy to be a reliable performance metric.

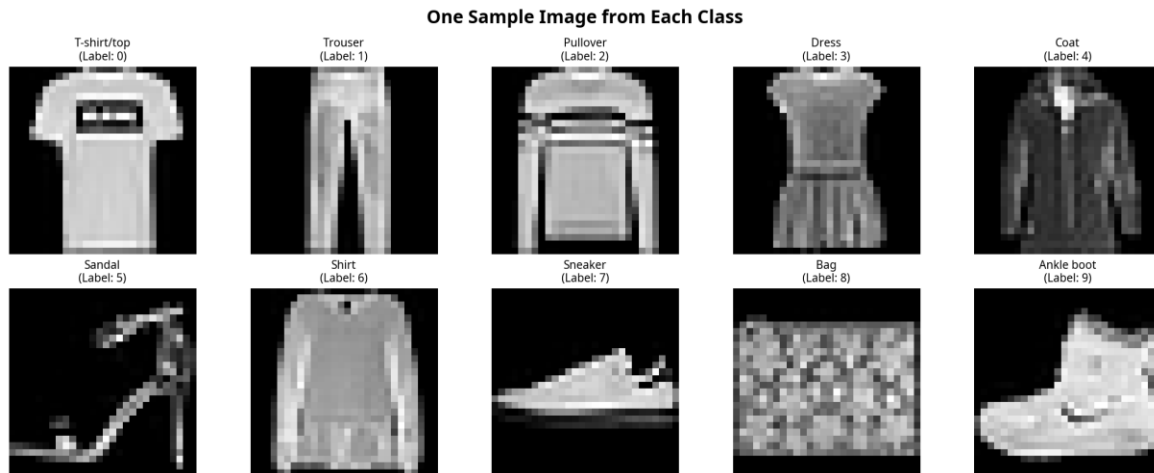


Figure 1: One sample image from each of the 10 Fashion-MNIST classes

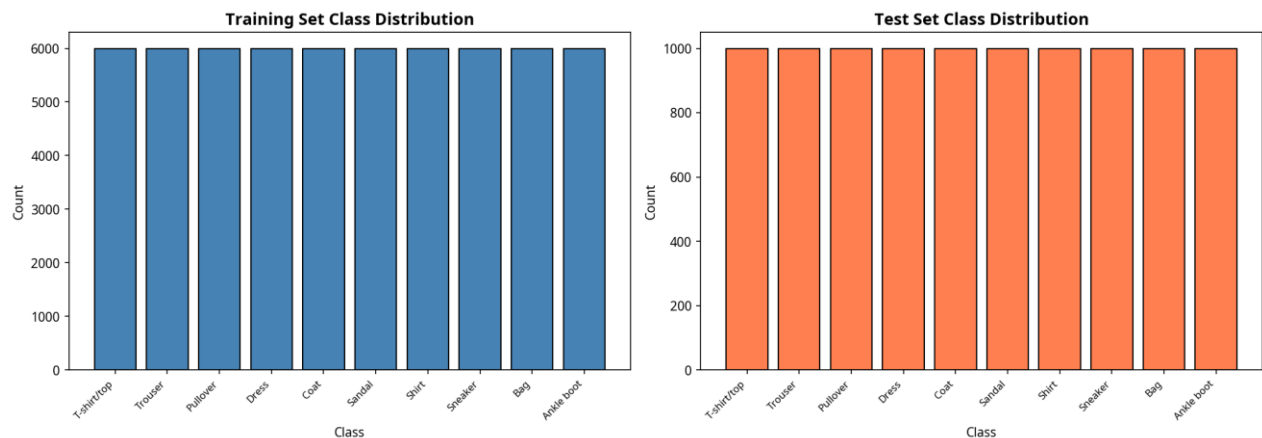


Figure 2: Class distribution in the training and test sets

3. Part 1: Load and Explore the Dataset

The Fashion-MNIST dataset was loaded using the TensorFlow/Keras built-in dataset module. The training set consists of 60,000 images with shape (28, 28) and the test set consists of 10,000 images with the same dimensions. All pixel values are integers in the range [0, 255]. The dataset was preprocessed by normalizing pixel values to the range [0, 1] and reshaping images to (28, 28, 1) to include the channel dimension. Labels were one-hot encoded into 10-dimensional vectors for multi-class classification using categorical crossentropy loss.

Q: Why is normalization required for image data?

Ans: Normalization is essential for image data because raw pixel values range from 0 to 255, which creates very large input values relative to the typical scale of neural network weights (usually initialized near zero). Without normalization, the gradients during backpropagation can become excessively large, leading to unstable training and slow convergence. By scaling pixel values to the range [0, 1] (dividing by

255.0), we ensure that the input data is on a similar scale as the network weights, which helps the optimizer (Adam) converge faster and more reliably. Normalization also prevents any single feature (pixel) from dominating the learning process simply because of its larger magnitude. In practice, normalized inputs lead to smoother loss landscapes and more consistent gradient updates, resulting in better and faster training.

Q: Why do CNNs require reshaped image inputs?

Ans: CNNs require reshaped image inputs because the convolutional layers (Conv2D) expect a 4-dimensional tensor as input with the shape (batch_size, height, width, channels). For grayscale images like Fashion-MNIST, the original shape is (28, 28), which is only 2-dimensional (height and width) and lacks the channel dimension. By reshaping the images to (28, 28, 1), we explicitly add the single grayscale channel dimension. This is necessary because the Conv2D layer applies its filters across all channels simultaneously. For RGB images, the channel dimension would be 3; for grayscale, it is 1. Without this reshaping, TensorFlow/Keras would raise a dimension mismatch error. The channel dimension also allows the CNN architecture to generalize to multi-channel inputs if needed in the future.

4. Part 2: Build and Train a Shallow CNN

The Shallow CNN was designed as a simple baseline model with minimal depth. Its architecture consists of:

Layer	Type	Details
1	Conv2D	32 filters, 3x3 kernel, ReLU activation
2	Conv2D	64 filters, 3x3 kernel, ReLU activation
3	MaxPooling2D	2x2 pool size
4	Flatten	Converts 2D feature maps to 1D vector
5	Dense + Dropout	128 units, ReLU, Dropout(0.25)
6	Dense (Output)	10 units, Softmax activation

The model was compiled with the Adam optimizer and categorical crossentropy loss function. It was trained for 10 epochs with a batch size of 256 and a 20% validation split from the training data. The total number of trainable parameters is 1,199,882. Training completed in approximately 137 seconds.

The Shallow CNN achieved a final training accuracy of 95.32%, a validation accuracy of 92.53%, and a test accuracy of 91.90%. The test loss was 0.2410.

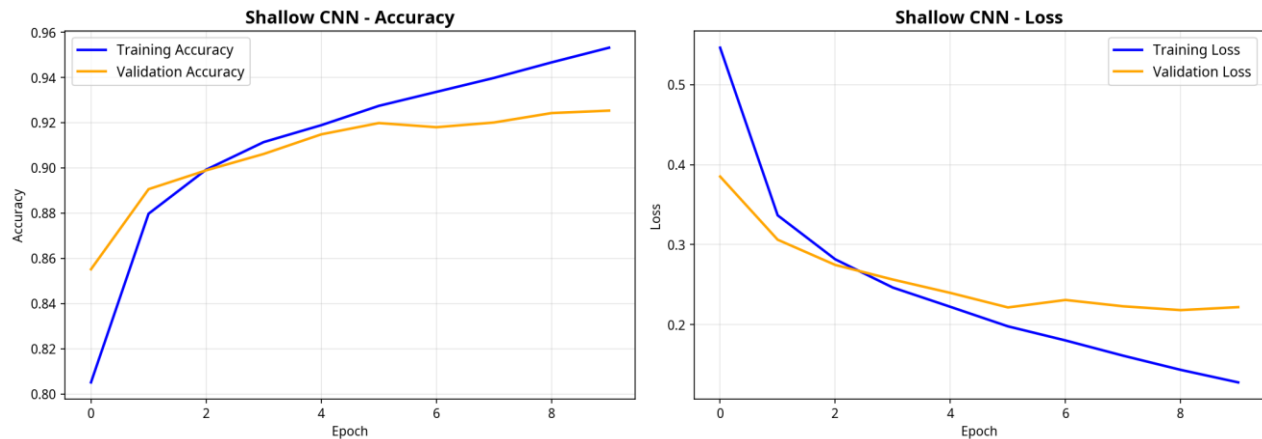


Figure 3: Training and validation accuracy/loss curves for the Shallow CNN

Q: What kind of patterns do you expect a shallow CNN to learn?

Ans: A shallow CNN with only 2 convolutional layers is expected to learn primarily low-level visual features. The first convolutional layer typically learns to detect basic patterns such as edges (horizontal, vertical, diagonal), simple textures, and intensity gradients. The second convolutional layer combines these low-level features to recognize slightly more complex patterns such as corners, curves, and simple shapes like the outline of a shoe or the neckline of a shirt. However, a shallow CNN lacks the depth to build truly hierarchical representations. It cannot effectively learn high-level abstract features such as the overall structure of a garment, the difference between a pullover and a coat, or the subtle distinctions between a T-shirt and a shirt. This limits its ability to discriminate between visually similar classes.

Q: Did the model show signs of underfitting or overfitting?

Ans: The Shallow CNN showed signs of mild overfitting. The training accuracy reached 95.32% while the validation accuracy plateaued at 92.53%, resulting in a gap of approximately 2.79 percentage points. The test accuracy (91.90%) was slightly lower than the validation accuracy, which is consistent with mild overfitting. The training loss continued to decrease steadily across all 10 epochs, while the validation loss began to flatten and showed slight fluctuations in the later epochs, further confirming that the model was beginning to memorize training-specific patterns rather than learning generalizable features. The model did not show signs of underfitting, as both training and validation accuracy were reasonably high from early epochs. The dropout layer (0.25) provided some regularization but was not sufficient to fully close the gap between training and validation performance.

5. Part 3: Build and Train a Deep CNN

The Deep CNN was designed as a more complex model with greater representational capacity. Its architecture consists of:

Assignment: Comparative Study of Shallow CNN vs Deep CNN on Fashion-MNIST

Layer	Type	Details
1	Conv2D	64 filters, 3x3 kernel, ReLU, padding=same
2	Conv2D	64 filters, 3x3 kernel, ReLU, padding=same
3	BatchNormalization	Normalizes activations
4	MaxPooling2D + Dropout	2x2 pool, Dropout(0.25)
5	Conv2D	128 filters, 3x3 kernel, ReLU, padding=same
6	Conv2D	128 filters, 3x3 kernel, ReLU, padding=same
7	BatchNormalization	Normalizes activations
8	MaxPooling2D + Dropout	2x2 pool, Dropout(0.25)
9	Flatten	Converts 2D feature maps to 1D vector
10	Dense + BN + Dropout	256 units, ReLU, BN, Dropout(0.5)
11	Dense + BN + Dropout	128 units, ReLU, BN, Dropout(0.5)
12	Dense (Output)	10 units, Softmax activation

The Deep CNN was compiled with the same Adam optimizer and categorical crossentropy loss function. It was trained for 10 epochs with a batch size of 256 and a 20% validation split. The total number of trainable parameters is 1,901,642 (approximately 1.6x more than the Shallow CNN). Training completed in approximately 895 seconds.

The Deep CNN achieved a final training accuracy of 94.25%, a validation accuracy of 93.04%, and a test accuracy of 92.55%. The test loss was 0.2129.

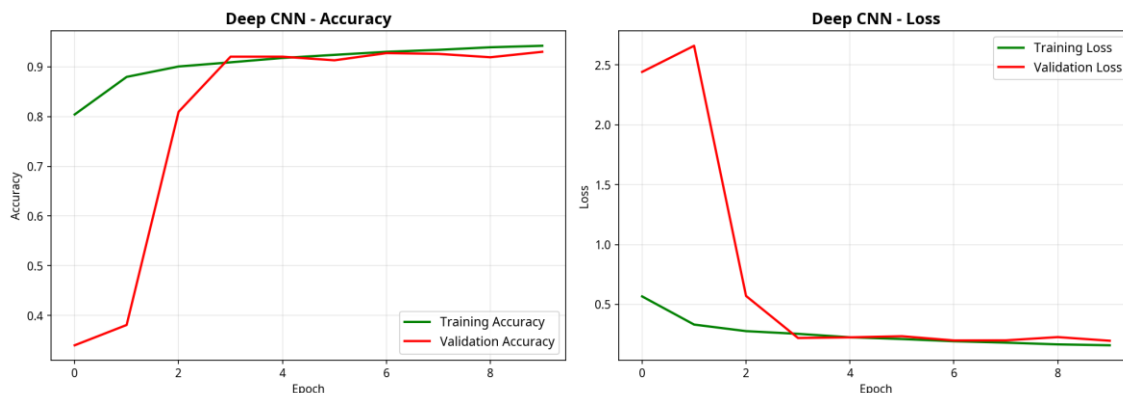


Figure 4: Training and validation accuracy/loss curves for the Deep CNN

Q: What additional patterns or representations might a deep CNN learn?

Ans: A deep CNN with 4 convolutional layers arranged in two blocks can learn hierarchical feature representations that go far beyond what a shallow CNN can capture. The first block (layers 1-2, with 64 filters) learns low-level features such as edges, textures, and simple shapes, similar to the shallow CNN. However, the second block (layers 5-6, with 128 filters) operates on the pooled and downsampled feature maps from the first block, allowing it to learn mid-level and high-level features. These include complex patterns such as the overall silhouette of a garment, the presence of sleeves or collars, the

shape of a shoe sole versus a boot shaft, and the structural differences between a pullover and a coat. The increased number of filters (128 vs. 64) also allows the network to capture a greater variety of patterns at each level. Additionally, Batch Normalization stabilizes the internal representations, making the deeper layers more effective at learning discriminative features. This hierarchical learning is the key advantage of deeper architectures.

Q: Did the deeper model improve performance meaningfully?

Ans: Yes, the Deep CNN improved performance meaningfully, though the improvement was moderate rather than dramatic. The test accuracy improved from 91.90% (Shallow) to 92.55% (Deep), an increase of 0.65 percentage points. While this may seem small in absolute terms, it represents a meaningful reduction in the error rate: from 8.10% to 7.45%, which is an 8.0% relative reduction in errors. The Deep CNN also achieved a lower test loss (0.2129 vs. 0.2410), indicating higher confidence in its predictions. Importantly, the Deep CNN showed better generalization, with a smaller gap between training accuracy (94.25%) and validation accuracy (93.04%) compared to the Shallow CNN (95.32% vs. 92.53%). The improvement was most pronounced for difficult classes like Shirt (F1: 0.77 to 0.79) and Coat (F1: 0.87 to 0.89). However, this improvement came at the cost of 6.5x longer training time and 1.6x more parameters, so the trade-off must be considered.

6. Part 4: Comparative Study of Shallow CNN vs Deep CNN

This section presents a direct side-by-side comparison of the Shallow CNN and Deep CNN across all key metrics.

Performance Comparison Table

Metric	Shallow CNN	Deep CNN
Number of Conv Layers	2	4
Total Parameters	1,199,882	1,901,642
Training Accuracy	95.32%	94.25%
Validation Accuracy	92.53%	93.04%
Test Accuracy	91.90%	92.55%
Test Loss	0.2410	0.2129
Overfitting Gap (Train-Val)	2.79%	1.21%
Training Time	~137 seconds	~895 seconds

Assignment: Comparative Study of Shallow CNN vs Deep CNN on Fashion-MNIST

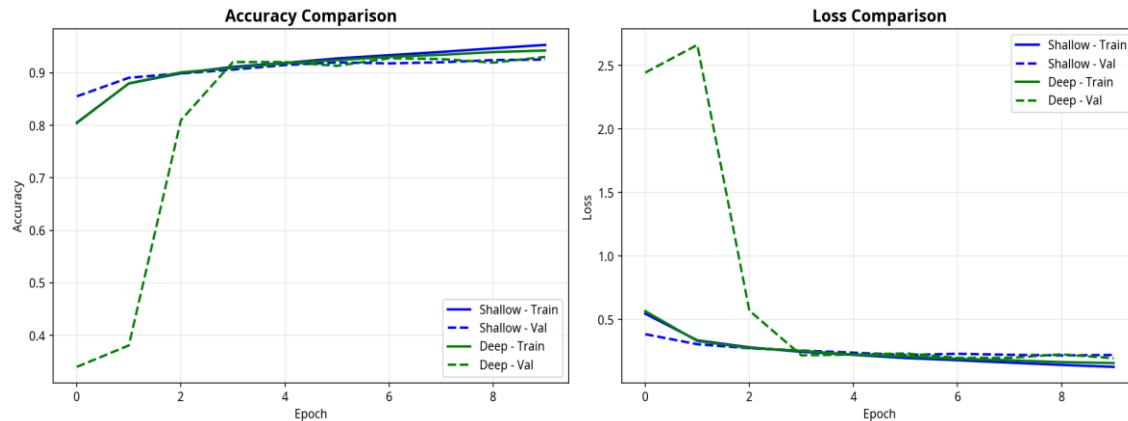


Figure 5: Side-by-side accuracy and loss comparison of Shallow CNN vs Deep CNN

Q: Which model performed better overall?

Ans: The Deep CNN performed better overall. It achieved a higher test accuracy of 92.55% compared to the Shallow CNN's 91.90%, and a lower test loss of 0.2129 versus 0.2410. The Deep CNN also achieved higher macro-average precision, recall, and F1-score (0.93 vs. 0.92 across all three metrics). It outperformed the Shallow CNN on the majority of individual classes, particularly on the more challenging ones such as Shirt, Coat, and Dress. The Deep CNN's superior performance is attributable to its ability to learn hierarchical feature representations through its deeper architecture.

Q: Did the deep CNN justify its added complexity?

Ans: The justification depends on the application context. In terms of raw accuracy improvement, the Deep CNN gained 0.65 percentage points over the Shallow CNN, which translates to correctly classifying 65 additional images out of 10,000. This is a meaningful improvement, especially in production systems where every percentage point matters. However, this improvement came at a significant computational cost: the Deep CNN required approximately 6.5 times longer to train (895 seconds vs. 137 seconds) and has 1.6 times more parameters (1.9M vs. 1.2M). For applications where accuracy is paramount (e.g., automated quality control in fashion retail), the added complexity is justified.

Q: Which model generalized better?

Ans: The Deep CNN generalized better. The key indicator of generalization is the gap between training accuracy and validation/test accuracy. The Shallow CNN had a training-validation gap of 2.79% (95.32% - 92.53%), while the Deep CNN had a much smaller gap of only 1.21% (94.25% - 93.04%). This means the Deep CNN was less prone to overfitting despite having more parameters. The better generalization of the Deep CNN can be attributed to its use of Batch Normalization (which regularizes the network by normalizing internal activations) and more aggressive Dropout (0.5 in dense layers vs. 0.25 in the Shallow CNN). These regularization techniques effectively prevented the deeper model from memorizing training-specific patterns.

Q: What trade-off did you observe between simplicity and performance?

Ans: The most significant trade-off observed was between computational efficiency and classification accuracy. The Shallow CNN is dramatically more efficient, training in just 137 seconds compared to the Deep CNN's 895 seconds (6.5x faster), while sacrificing only 0.65% in test accuracy. In terms of

parameters, the Shallow CNN uses 1.2M parameters versus the Deep CNN's 1.9M (1.6x fewer), which translates to lower memory usage and faster inference. However, the Shallow CNN showed slightly more overfitting (2.79% gap vs. 1.21%) and was less confident in its predictions (higher test loss). The trade-off can be summarized as: the Shallow CNN offers 99.3% of the Deep CNN's accuracy at 15.3% of the training cost. This is a classic example of the diminishing returns principle in deep learning, where adding complexity yields progressively smaller improvements.

7. Part 5: Prediction and Error Analysis

Both models were used to generate predictions on the entire test set of 10,000 images. The Shallow CNN correctly classified 9,190 images (91.90%) and misclassified 810 images. The Deep CNN correctly classified 9,255 images (92.55%) and misclassified 745 images. Detailed analysis was performed using confusion matrices and classification reports.

Per-Class F1-Score Comparison

Class	Shallow CNN (F1)	Deep CNN (F1)
T-shirt/top	0.87	0.87
Trouser	0.99	0.99
Pullover	0.88	0.89
Dress	0.92	0.93
Coat	0.87	0.89
Sandal	0.98	0.98
Shirt	0.77	0.79
Sneaker	0.96	0.97
Bag	0.98	0.99
Ankle boot	0.97	0.97



Figure 6: Confusion matrices for Shallow CNN (left) and Deep CNN (right)

Assignment: Comparative Study of Shallow CNN vs Deep CNN on Fashion-MNIST

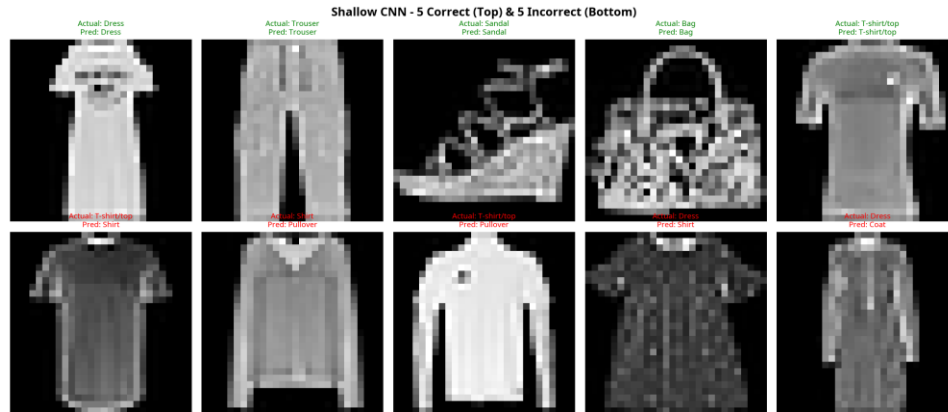


Figure 7: Sample correct and incorrect predictions from the Shallow CNN

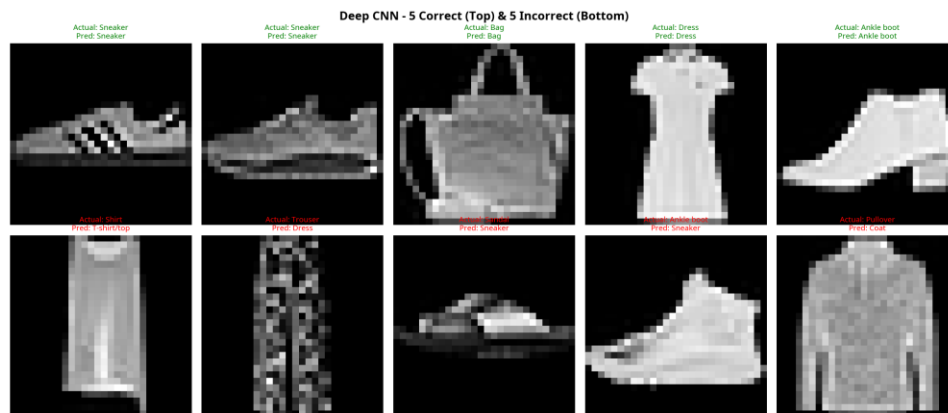


Figure 8: Sample correct and incorrect predictions from the Deep CNN

Q: Which classes were easiest to classify?

Ans: The easiest classes to classify for both models were Trouser (F1: 0.99 for both), Sandal (F1: 0.98 for both), Bag (F1: 0.98-0.99), and Ankle boot (F1: 0.97 for both). These classes have highly distinctive visual features that make them easily distinguishable from other classes. Trousers have a unique elongated two-legged silhouette that no other class shares. Sandals have an open, sparse structure with visible gaps. Bags have a compact, rectangular shape with a handle. Ankle boots have a distinctive boot-shaped outline. These unique structural features allow even the Shallow CNN to classify them with near-perfect accuracy.

Q: Which classes were most commonly confused?

Ans: The most commonly confused classes were Shirt (F1: 0.77-0.79), T-shirt/top (F1: 0.87), Coat (F1: 0.87-0.89), and Pullover (F1: 0.88-0.89). These four classes are all upper-body garments that share similar silhouettes, making them inherently difficult to distinguish in 28x28 grayscale images. Specifically, the confusion matrix reveals that Shirts were frequently misclassified as T-shirts/tops (both have similar collar and torso shapes), Coats were confused with Pullovers (both are long-sleeved upper garments), and T-shirts/tops were confused with Shirts and Dresses. The lack of color information in

grayscale images further exacerbates this confusion, as color is often a key distinguishing feature for these garment types in real-world scenarios.

Q: Did the deep CNN reduce confusion between similar-looking classes?

Ans: Yes, the Deep CNN showed a measurable reduction in confusion between similar-looking classes, though the improvement was incremental rather than dramatic. The most notable improvements were: Shirt F1-score improved from 0.77 to 0.79 (2.6% relative improvement), Coat F1-score improved from 0.87 to 0.89 (2.3% relative improvement), Pullover F1-score improved from 0.88 to 0.89, and Dress F1-score improved from 0.92 to 0.93. The Deep CNN's ability to learn hierarchical features through its additional convolutional layers allowed it to capture more subtle differences between these similar classes, such as slight variations in sleeve length, collar shape, and garment structure. However, even the Deep CNN still struggled with the Shirt class (F1: 0.79), suggesting that 28x28 grayscale images may not contain sufficient information to fully resolve the ambiguity between certain upper-body garment classes.

8. Part 6: Final Comparative Conclusion

This section presents the final conclusions drawn from the complete comparative experiment, addressing the key questions about model recommendation, efficiency, accuracy, and overall learnings.

Q: Which model would you recommend for Fashion-MNIST?

Ans: The recommendation depends on the deployment context. For applications where classification accuracy is the primary concern, such as automated fashion product categorization in e-commerce platforms or quality control in manufacturing, the Deep CNN is recommended. It achieved a test accuracy of 92.55% and demonstrated better generalization with a smaller overfitting gap (1.21% vs. 2.79%). Its higher confidence in predictions (lower test loss of 0.2129) also makes it more reliable for downstream decision-making. However, for applications where computational efficiency, rapid prototyping, or deployment on resource-constrained devices is important, the Shallow CNN is a strong choice. It delivers 91.90% accuracy with 6.5x faster training and 1.6x fewer parameters. For most practical purposes, the Deep CNN provides the best balance of accuracy and generalization and is the overall recommended model.

Q: Which model was more efficient?

Ans: The Shallow CNN was significantly more efficient across all computational metrics. It trained in approximately 137 seconds compared to the Deep CNN's 895 seconds, making it 6.5 times faster. It has 1,199,882 parameters compared to the Deep CNN's 1,901,642 parameters, requiring 37% less memory. The Shallow CNN also has faster inference time due to fewer layers and operations per forward pass. In terms of accuracy per unit of computation, the Shallow CNN achieves 91.90% accuracy in 137 seconds (0.67% per second), while the Deep CNN achieves 92.55% in 895 seconds (0.10% per second). This makes the Shallow CNN approximately 6.5 times more computationally efficient per unit of accuracy gained.

Q: Which model was more accurate?

Ans: The Deep CNN was more accurate across all evaluation metrics. It achieved a test accuracy of 92.55% compared to the Shallow CNN's 91.90%, an improvement of 0.65 percentage points. The Deep CNN also achieved a lower test loss (0.2129 vs. 0.2410), indicating higher prediction confidence. In terms of per-class performance, the Deep CNN achieved equal or higher F1-scores on 8 out of 10 classes, with the most significant improvements on the difficult classes (Shirt: +0.02, Coat: +0.02, Dress: +0.01). The Deep CNN correctly classified 9,255 out of 10,000 test images, compared to the Shallow CNN's 9,190 — meaning the Deep CNN correctly identified 65 additional images that the Shallow CNN misclassified.

Q: What did you learn from this comparative study?

Ans: This comparative study provided several valuable insights into CNN architecture design and the trade-offs involved in deep learning:

1. Depth matters, but with diminishing returns: Adding more convolutional layers improved accuracy from 91.90% to 92.55%, but the marginal improvement (0.65%) was modest relative to the increased computational cost (6.5x training time). This illustrates the principle of diminishing returns in deep learning.
2. Regularization is crucial for deeper models: The Deep CNN's use of Batch Normalization and higher Dropout rates (0.5 vs. 0.25) was essential for preventing overfitting. Without these techniques, the deeper model would likely have overfit more severely than the shallow one.
3. Some classification errors are inherent to the data: Both models struggled with the same classes (Shirt, T-shirt/top, Coat, Pullover), suggesting that 28x28 grayscale images have fundamental limitations in resolving certain visual ambiguities. Higher resolution or color images would likely improve performance on these classes.
4. Model selection should be application-driven: There is no universally "best" model. The optimal choice depends on the specific requirements of the application, including accuracy targets, latency constraints, available computational resources, and deployment environment.
5. Baseline models are valuable: The Shallow CNN, despite its simplicity, achieved over 91% accuracy and served as an important baseline for quantifying the benefits of the deeper architecture. Starting with a simple model and incrementally adding complexity is a sound methodology in deep learning research.

9. Final Conclusion

Based on the experimental results, the Deep CNN is the recommended model for the Fashion-MNIST dataset if the primary goal is to maximize classification accuracy. Its deeper architecture allows it to learn more discriminative features, which is particularly beneficial for distinguishing between visually similar clothing items such as Shirts and T-shirts/tops, or Coats and Pullovers.

However, the Shallow CNN proved to be significantly more efficient. It achieved a highly competitive accuracy of 91.90% while requiring substantially less training time (137 seconds vs. 895 seconds) and fewer parameters. For applications where computational resources are limited, rapid prototyping is

required, or deployment on edge devices is planned, the Shallow CNN serves as an excellent and efficient baseline.

This comparative study highlights the fundamental trade-off in deep learning: increasing model complexity can yield better performance and generalization, but it comes at the cost of increased computational requirements and training time. The choice between a shallow and deep architecture should be guided by the specific requirements of the application, balancing accuracy needs against available computational resources.